

DBMS Notes — Deepfake Detection Project

Beginner-friendly notes on how this project stores its data: which database it uses, what the tables look like, and every SQL command the code runs.

All the database code lives in the `backend/` folder:

File	What it does with the database
<code>backend/database.py</code>	Creates the tables, opens connections, runs queries
<code>backend/config.py</code>	Reads the database connection string from <code>.env</code>
<code>backend/main.py</code>	Calls <code>init_schema()</code> on startup so the tables exist
<code>backend/routes/analyze.py</code>	<code>INSERT</code> a new video analysis
<code>backend/routes/history.py</code>	<code>SELECT</code> / <code>UPDATE</code> analyses, and the combined history list
<code>backend/routes/live.py</code>	<code>INSERT</code> / <code>SELECT</code> live screen-share sessions

1. The big picture

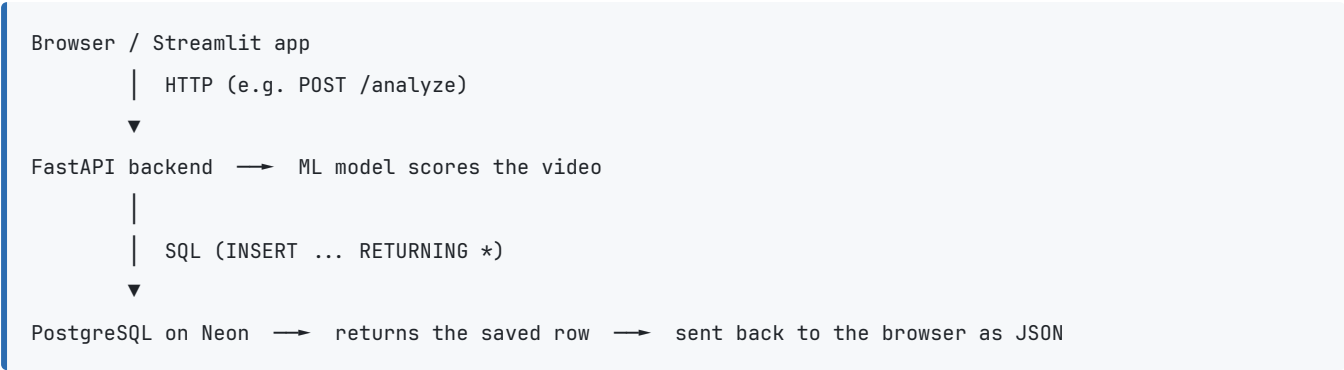
- **DBMS used:** PostgreSQL (often just called "Postgres"). It is a *relational* database: data lives in tables made of rows and columns.
- **Where it runs:** [Neon](#), a free cloud-hosted Postgres. Locally you can also run a normal Postgres server instead.
- **How Python talks to it:** the `psycopg2` library (a Postgres *driver*).
- **No ORM:** every query is hand-written SQL. (An ORM like SQLAlchemy would let you write Python classes instead of SQL — this project does not use one, which makes it a good place to read real SQL.)
- **No migration tool:** the tables are created automatically when the server starts (see section 4).

What goes in the database, and what does not

Stored in the database	Stored somewhere else
Result of every uploaded video analysis	The actual video file — saved on disk in <code>backend/uploads/</code> ; the DB only keeps its path
Summary of every finished live screen-share session	A live session that is still running — kept in the server's memory until you press stop
User feedback on a result	The ML model — a <code>.pth</code> file on disk

Rule of thumb used here: **big binary files go on disk, the database stores small structured facts about them.**

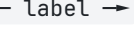
How a request flows



Data flow diagram (DFD)

A **DFD** shows where data comes from, which process handles it, and where it is stored. It says nothing about the order things happen in (that's a flowchart's job).

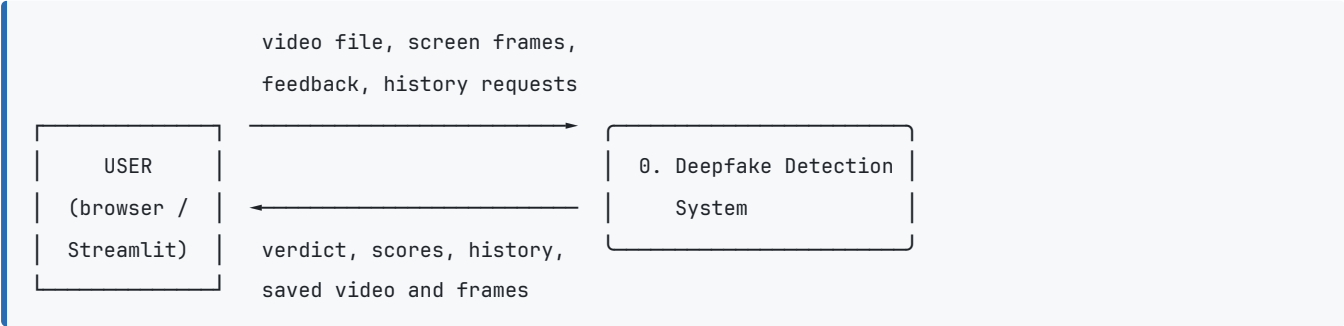
Notation used below:

Symbol	Meaning
 square box	External entity — someone outside the system (the user)
 rounded box	Process — something the system does to the data
 D1 name	Data store — where data rests (a table, a folder, memory)
 label →	Data flow — the arrow is labelled with the data that moves

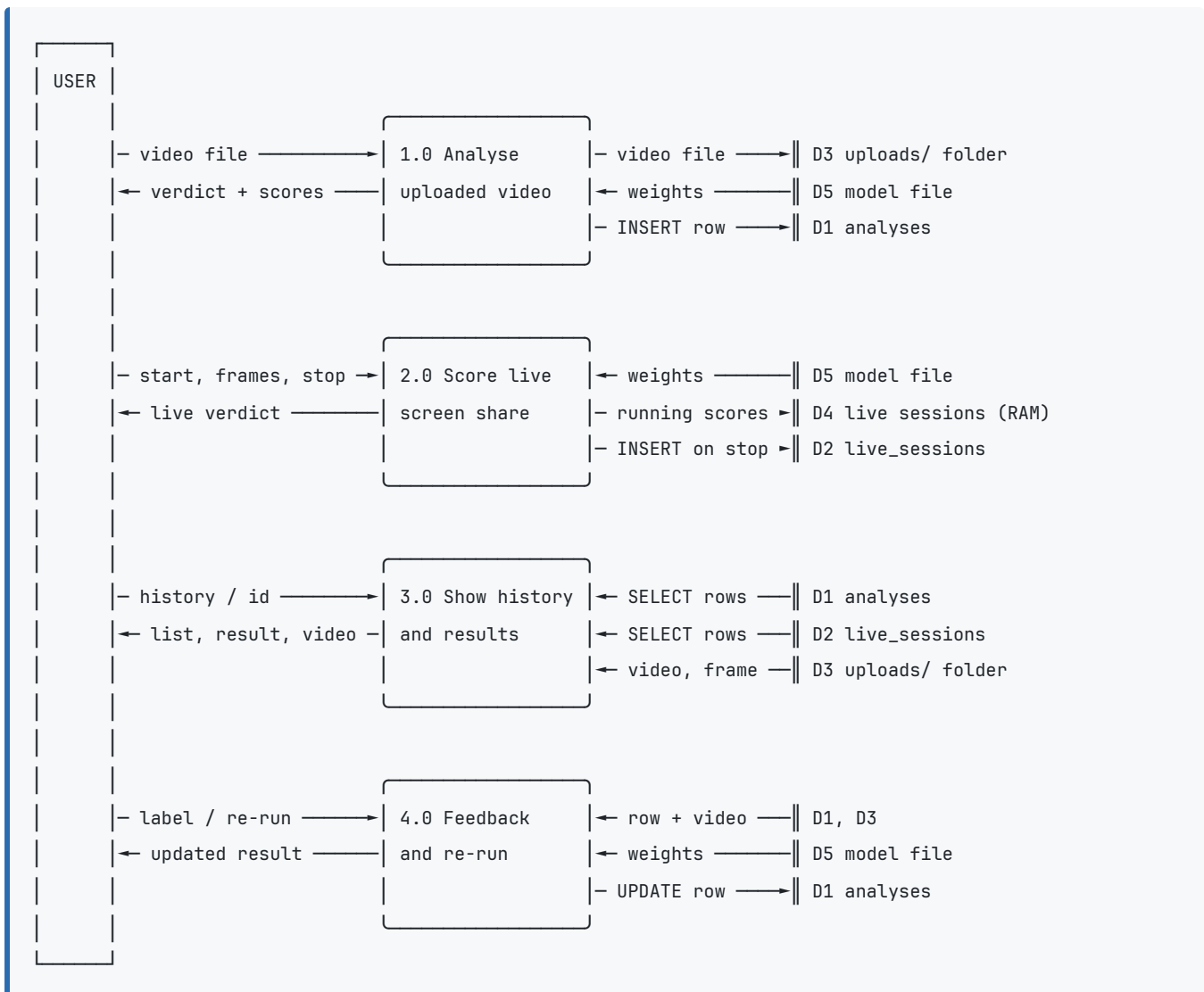
The data stores in this project:

Store	What it is
D1 analyses	Postgres table (section 3.1)
D2 live_sessions	Postgres table (section 3.2)
D3 uploads/ folder	Uploaded video files on disk
D4 live sessions (RAM)	Sessions still running, kept in server memory until stop
D5 model file	The trained .pth checkpoint the model loads its weights from

Level 0 (context diagram) — the whole system as one process:



Level 1 — the system opened up into its four processes. A store drawn twice is the same store; it is repeated only to avoid crossing lines.



Process	Endpoints	Reads	Writes
1.0 Analyse uploaded video	POST /analyze	D5	D3 (video file), D1 (INSERT)
2.0 Score live screen share	POST /live/start , /live/{id}/frame , /live/{id}/stop	D5, D4	D4 (running scores), D2 (INSERT on stop)
3.0 Show history and results	GET /history , GET /analysis/{id} (+ /video , /frame), GET /live/sessions/{id}	D1, D2, D3	—
4.0 Feedback and re-run	POST /analysis/{id}/feedback , POST /analysis/{id}/rerun	D1, D3, D5	D1 (UPDATE)

Notice that only D1 and D2 are in the database. D3 and D5 are files on disk, and D4 disappears when the server restarts — which is exactly why a live session is copied into D2 when the user presses stop.

2. Connecting to the database

The connection details are one string, called a **connection string** or **URL**, stored in `backend/.env` :

```
NEON_DB_URL=postgresql://USER:PASSWORD@HOST/dbname?sslmode=require
```

Reading it piece by piece:

Part	Meaning
postgresql://	Which kind of database
USER:PASSWORD	Login details
HOST	The server address (Neon gives you this)
dbname	Which database on that server
sslmode=require	Encrypt the connection

`DATABASE_URL` works as another name for the same setting. The `.env` file is **never committed to git** because it contains the password — only `.env.example` (with placeholders) is.

For a local Postgres instead of Neon:

```
NEON_DB_URL=postgresql://postgres:devpass@127.0.0.1:55432/deepfake
```

3. The tables (schema)

A **schema** is the design of the database: which tables exist, what columns they have, and what type of data each column holds.

The project has **2 tables**:

1. `analyses` — one row per uploaded video
2. `live_sessions` — one row per finished live screen-share

3.1 Table `analyses`

One row = one uploaded video that was checked for deepfakes.

Column	Type	Rules	Meaning
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>	Unique number for the row, auto-generated (1, 2, 3, ...)
<code>filename</code>	<code>TEXT</code>	<code>NOT NULL</code>	Original name of the uploaded file, e.g. <code>clip.mp4</code>
<code>storage_path</code>	<code>TEXT</code>	can be NULL	Where the video is saved on disk
<code>fake_probability</code>	<code>DOUBLE PRECISION</code>	<code>NOT NULL</code>	Model's score from 0.0 (real) to 1.0 (fake)
<code>status</code>	<code>TEXT</code>	<code>NOT NULL</code>	Verdict label: <code>REAL</code> , <code>SUSPICIOUS</code> or <code>HIGH_RISK</code>

Column	Type	Rules	Meaning
suspicious_start	DOUBLE PRECISION	can be NULL	Second where the suspicious part begins
suspicious_end	DOUBLE PRECISION	can be NULL	Second where the suspicious part ends
frame_scores	JSONB	NOT NULL	List of per-frame scores (see 3.3)
user_feedback	TEXT	can be NULL	What the user said about the result
created_at	TIMESTAMPTZ	NOT NULL , DEFAULT now()	When the row was created, filled in automatically

Example row:

id	filename	fake_probability	status	suspicious_start	suspicious_end	created_at
1	006_002.mp4	0.9733	HIGH_RISK	0.0	10.2	2026-09-01 20:45:09+00

3.2 Table live_sessions

One row = one live screen-share session, saved when the user presses stop.

Column	Type	Rules	Meaning
id	SERIAL	PRIMARY KEY	Unique auto-generated number
started_at	TIMESTAMPTZ	NOT NULL	When the share began
ended_at	TIMESTAMPTZ	NOT NULL	When it stopped
duration_seconds	DOUBLE PRECISION	NOT NULL	How long it lasted
frames_scored	INTEGER	NOT NULL	How many frames had a face and were scored
mean_probability	DOUBLE PRECISION	NOT NULL	Average fake score over the session
peak_probability	DOUBLE PRECISION	NOT NULL	Highest fake score seen
status	TEXT	NOT NULL	Final verdict label
timeline	JSONB	NOT NULL	Scores over time (same shape as frame_scores)
user_feedback	TEXT	can be NULL	User's feedback

3.3 What the JSONB columns hold

frame_scores and timeline store a list inside a single cell, as JSON:

```
[
  {"timestamp": 0.0, "fake_probability": 0.91},
  {"timestamp": 0.2, "fake_probability": 0.95},
  {"timestamp": 0.4, "fake_probability": 0.97}
]
```

In a strictly "by the book" design this list would be its own table (`frame_scores(analysis_id, timestamp, fake_probability)`). The project stores it as JSON instead because the list is always read and written **all at once, together with its parent row** — never searched or updated one frame at a time. That keeps things to one simple query.

3.4 How the two tables relate

They **don't** — there is no foreign key between them. They are two independent kinds of record that happen to be shown together on the History screen (see query 5.6).



Because the ids come from two separate tables, `id = 5` in `analyses` and `id = 5` in `live_sessions` are **different things**. That's why the history list adds a `kind` column to tell them apart.

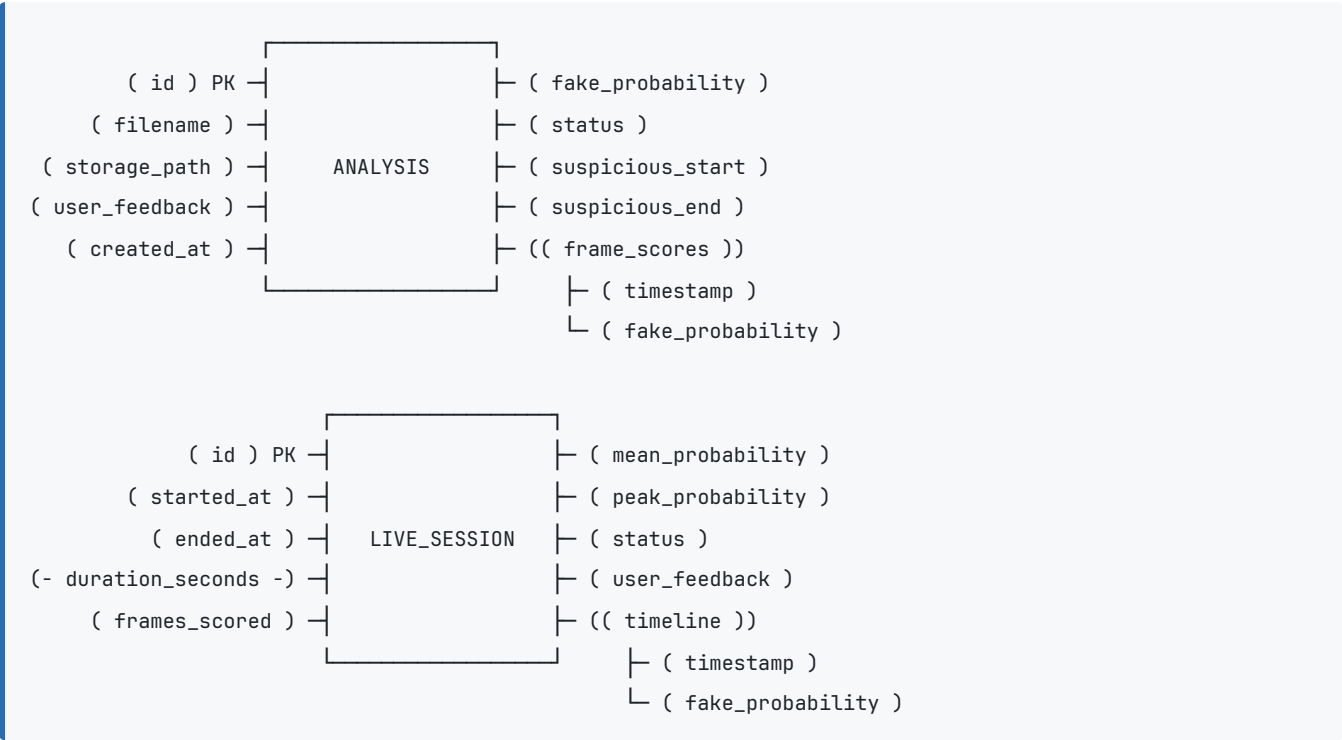
3.5 ER diagram

An **ER (Entity–Relationship) diagram** is the *conceptual* design: the things the system keeps data about (**entities**), their properties (**attributes**), and how they are connected (**relationships**). The tables in 3.1 and 3.2 are what this diagram turns into.

Notation (Chen style, drawn in plain text):

Symbol	Meaning
⎓ box	Entity — becomes a table
(name)	Attribute — becomes a column
(name) PK	Key attribute — underlined in a hand-drawn ER diagram; becomes the primary key
((name))	Multivalued attribute — holds a list of values (double oval)
└ (part) under an attribute	Composite attribute — the parts it is made of
(- name -)	Derived attribute — can be calculated from other attributes (dashed oval)
< name >	Relationship (diamond)

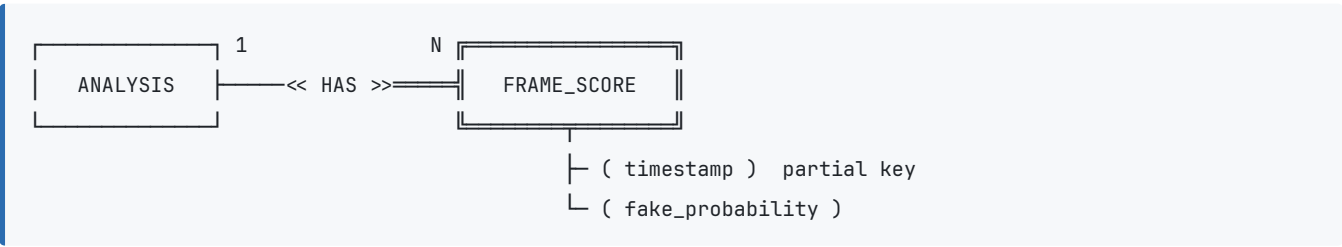
Symbol	Meaning
<< name >>, 	Identifying relationship / weak entity (double diamond / double box)



Reading it:

- **Two strong entities**, ANALYSIS and LIVE_SESSION . Each has its own key (id), so neither depends on anything else to exist.
- **No relationship diamond** between them — the same "no foreign key" point as 3.4.
- frame_scores / timeline are **multivalued** (one analysis has many frame scores) and **composite** (each score is a timestamp plus a fake_probability). The textbook way to map a multivalued attribute is a separate table; this project stores it as JSONB instead (see 3.3).
- duration_seconds is **derived**: it is ended_at - started_at . The project still stores it as a column — a small, deliberate redundancy that saves recalculating it on every read.

The "by the book" alternative. If the per-frame scores were normalized into their own table, FRAME_SCORE would become a **weak entity**: it has no key of its own and only makes sense attached to one analysis. Double lines mark the weak entity, its **identifying relationship**, and its **total participation** (every frame score must belong to an analysis):



- **1 : N** cardinality — one analysis, many frame scores.
- timestamp is a **partial key** (dashed underline by hand): unique only *within* one analysis. The full key is (analysis_id, timestamp) .
- Mapped to tables, this becomes frame_scores(analysis_id FK → analyses.id, timestamp, fake_probability) with PRIMARY KEY (analysis_id, timestamp) — the table mentioned in 3.3.

4. Creating the tables (DDL)

DDL = *Data Definition Language*: commands that create or change the *structure* of the database (`CREATE` , `ALTER` , `DROP`).

When the backend starts, `main.py` calls `init_schema()` in `database.py` , which runs:

```
CREATE TABLE IF NOT EXISTS analyses (
    id                SERIAL PRIMARY KEY,
    filename           TEXT NOT NULL,
    storage_path       TEXT,
    fake_probability   DOUBLE PRECISION NOT NULL,
    status             TEXT NOT NULL,
    suspicious_start   DOUBLE PRECISION,
    suspicious_end     DOUBLE PRECISION,
    frame_scores       JSONB NOT NULL,
    user_feedback      TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS live_sessions (
    id                SERIAL PRIMARY KEY,
    started_at        TIMESTAMPTZ NOT NULL,
    ended_at          TIMESTAMPTZ NOT NULL,
    duration_seconds   DOUBLE PRECISION NOT NULL,
    frames_scored      INTEGER NOT NULL,
    mean_probability   DOUBLE PRECISION NOT NULL,
    peak_probability   DOUBLE PRECISION NOT NULL,
    status            TEXT NOT NULL,
    timeline           JSONB NOT NULL,
    user_feedback      TEXT
);

-- Columns added later; safe to run on an older database
ALTER TABLE analyses ADD COLUMN IF NOT EXISTS storage_path TEXT;
ALTER TABLE analyses ADD COLUMN IF NOT EXISTS user_feedback TEXT;
```

Why `IF NOT EXISTS` ? The server runs these every time it starts. Without it, the second start would fail with "table already exists". With it, the command does nothing when the table (or column) is already there. This is what lets the project skip a migration tool.

The two `ALTER TABLE` lines are a tiny hand-made "migration": databases created before those columns existed get them added; new databases already have them, so nothing happens.

5. Every query the app runs (DML)

DML = *Data Manipulation Language*: commands that work with the *data* inside the tables (`SELECT` , `INSERT` , `UPDATE` , `DELETE`).

The `%s` in each query is a **placeholder**. Python passes the real values separately and `psycopg2` fills them in safely (see section 6.1).

5.1 Save a new video analysis — `POST /analyze`

File: `backend/routes/analyze.py`

```
INSERT INTO analyses (  
    filename, storage_path, fake_probability, status,  
    suspicious_start, suspicious_end, frame_scores  
) VALUES (%s, %s, %s, %s, %s, %s, %s)  
RETURNING *;
```

- `id` and `created_at` are not listed — Postgres fills them in by itself (`SERIAL` and `DEFAULT now()`).
- `RETURNING *` is a Postgres feature: the `INSERT` hands back the full new row (including the new `id`), so no second `SELECT` is needed.

5.2 Get one analysis — `GET /analysis/{id}`

File: `backend/routes/history.py`

```
SELECT * FROM analyses WHERE id = %s;
```

Returns 0 or 1 rows. If 0, the API replies with **404 Not Found**. Looking up by `id` is fast because a `PRIMARY KEY` automatically gets an index.

5.3 Save user feedback — `POST /analysis/{id}/feedback`

```
UPDATE analyses SET user_feedback = %s WHERE id = %s RETURNING *;
```

Always keep the `WHERE` clause on an `UPDATE` — without it, **every row** in the table would be changed.

5.4 Re-run the model on a stored video — `POST /analysis/{id}/rerun`

```
UPDATE analyses SET  
    fake_probability = %s, status = %s,  
    suspicious_start = %s, suspicious_end = %s, frame_scores = %s  
WHERE id = %s  
RETURNING *;
```

Overwrites the old results of that one row with the new model's results.

5.5 Save a finished live session — POST /live/{session_id}/stop

File: backend/routes/live.py

```
INSERT INTO live_sessions (
    started_at, ended_at, duration_seconds, frames_scored,
    mean_probability, peak_probability, status, timeline
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
RETURNING *;
```

Read one back — GET /live/sessions/{id}:

```
SELECT * FROM live_sessions WHERE id = %s;
```

5.6 History list (both tables together) — GET /history

File: backend/routes/history.py . This is the most interesting query:

```
SELECT * FROM (
    SELECT id, 'upload' AS kind, filename, storage_path,
           fake_probability, status, created_at
    FROM analyses
    UNION ALL
    SELECT id, 'live' AS kind, 'Live screen share' AS filename,
           NULL AS storage_path,
           mean_probability AS fake_probability, status,
           ended_at AS created_at
    FROM live_sessions
) combined
ORDER BY created_at DESC, kind DESC, id DESC
LIMIT %s;
```

How it works, step by step:

1. **First SELECT** takes rows from `analyses` and adds a made-up column `kind` with the fixed text `'upload'`.
2. **Second SELECT** takes rows from `live_sessions` and reshapes them to look the same: a fixed filename, `NULL` for the missing path, and `AS` to **rename** columns (`mean_probability` → `fake_probability`, `ended_at` → `created_at`).
3. **UNION ALL** stacks the two results on top of each other.
 - Both sides must have the **same number of columns, in the same order, with compatible types**. That's why the second side invents columns.
 - `UNION ALL` keeps every row. Plain `UNION` would also remove duplicate rows, which costs extra work and isn't needed here.
4. The stacked result is wrapped in `( ... ) combined` — a **subquery** (also called a *derived table*) named `combined`, so it can be sorted as one.
5. **ORDER BY** `created_at DESC` puts newest first. `kind DESC`, `id DESC` are tie-breakers so the order is always the same when two rows share a time.
6. **LIMIT %s** returns only the first N rows (default 100).

Summary: which endpoint runs which SQL

API endpoint	SQL command	Table
POST /analyze	INSERT ... RETURNING *	analyses
GET /analysis/{id}	SELECT ... WHERE id =	analyses
GET /analysis/{id}/video	SELECT ... WHERE id = (to find the file path)	analyses
GET /analysis/{id}/frame	SELECT ... WHERE id =	analyses
POST /analysis/{id}/feedback	UPDATE ... RETURNING *	analyses
POST /analysis/{id}/rerun	SELECT , then UPDATE ... RETURNING *	analyses
POST /live/{id}/stop	INSERT ... RETURNING *	live_sessions
GET /live/sessions/{id}	SELECT ... WHERE id =	live_sessions
GET /history	SELECT ... UNION ALL ... ORDER BY ... LIMIT	both

The app never runs `DELETE` and never runs `DROP`.

6. How Python runs the SQL (`database.py`)

Three small helper functions do all the work:

Function	Use it for	Returns
<code>execute(query, params)</code>	Commands with no result (<code>CREATE</code> , <code>ALTER</code>)	nothing
<code>fetch_one(query, params)</code>	One row (<code>SELECT ... WHERE id</code> , <code>INSERT/UPDATE ... RETURNING</code>)	a dict, or <code>None</code>
<code>fetch_all(query, params)</code>	Many rows (history list)	a list of dicts

Each one follows the same steps:

```
conn = psycopg2.connect(DATABASE_URL)  # 1. open a connection
cur = conn.cursor(cursor_factory=RealDictCursor)  # 2. get a cursor
cur.execute(query, params)                # 3. run the SQL
row = cur.fetchone()                      # 4. read the result
cur.close()
conn.commit()                             # 5. save the changes
conn.close()                             # 6. close the connection (in `finally`)
```

- **Connection** — the link between Python and the database server.
- **Cursor** — the object you send a query through and read results from.
- **RealDictCursor** — makes each row come back as a dictionary (`{"id": 1, "filename": "clip.mp4", ...}`) instead of a plain tuple, so the code can use `row["filename"]` instead of `row[1]`.
- **commit()** — makes the changes permanent (see 6.2).
- **finally: conn.close()** — the connection is closed even if the query crashes, so connections don't leak.

- `Json(value)` (`database.to_json`) — wraps a Python list/dict so `psycopg2` sends it as a JSON value for a `JSONB` column.

A new connection is opened for every query. That is simple but slower than reusing connections (a *connection pool*). It's fine for this project's traffic.

6.1 Parameterized queries (protection from SQL injection)

The code **never** glues user input into SQL text. It does this:

```
# SAFE: value passed separately, psycopg2 escapes it
fetch_one("SELECT * FROM analyses WHERE id = %s", (analysis_id,))
```

and never this:

```
# UNSAFE: user input becomes part of the SQL command
fetch_one(f"SELECT * FROM analyses WHERE id = {analysis_id}")
```

If a user sent `1; DROP TABLE analyses` as input, the unsafe version would run it as SQL. That attack is called **SQL injection**. With placeholders, the input is always treated as a *value*, never as a command.

Note: `(analysis_id,)` has a trailing comma because the params must be a tuple, even for one value.

6.2 Transactions and commit

A **transaction** is a group of changes that either all happen or none do. `psycopg2` starts one automatically on the first query; `conn.commit()` saves it. If the connection closes without `commit()`, the changes are thrown away (**rolled back**).

That's why even `fetch_one` calls `commit()` — it's also used for `INSERT ... RETURNING` and `UPDATE ... RETURNING`, which change data.

This is the **A** (Atomicity) and **D** (Durability) of **ACID**:

Letter	Meaning	In this project
Atomicity	All or nothing	An analysis row is saved completely or not at all
Consistency	Rules are always respected	<code>NOT NULL</code> and <code>PRIMARY KEY</code> are always enforced
Isolation	Parallel transactions don't mess each other up	Two uploads at once get different <code>id</code> s
Durability	Once committed, it stays	Data survives a server restart (it's on Neon)

7. Data types and constraints used

Data types

Type	Holds	Example
<code>SERIAL</code>	Auto-increasing integer (shortcut for <code>INTEGER</code> + a sequence)	1, 2, 3

Type	Holds	Example
INTEGER	Whole number	52
DOUBLE PRECISION	Decimal number (8-byte float)	0.9733
TEXT	Any-length string	'HIGH_RISK'
TIMESTAMPZ	Date + time + time zone	2026-09-01 20:45:09+00
JSONB	JSON stored in a binary, searchable form	[{"timestamp": 0.0, ...}]

JSON vs JSONB : JSON stores the text exactly as given; JSONB parses it into a binary form that is faster to query and can be indexed. JSONB is the usual choice.

TIMESTAMP vs TIMESTAMPZ : TIMESTAMPZ remembers the time zone, so times stay correct if the server and user are in different zones.

Constraints

A **constraint** is a rule the database enforces on the data.

Constraint	Meaning	Used on
PRIMARY KEY	Unique and not null; identifies each row. Gets an index automatically	id in both tables
NOT NULL	The column must have a value	Most columns
DEFAULT now()	If no value is given, use the current time	analyses.created_at

Not used in this project (good to know): FOREIGN KEY , UNIQUE , CHECK . For example status could have had CHECK (status IN ('REAL', 'SUSPICIOUS', 'HIGH_RISK')) — right now the Python code is what makes sure only those values are written.

What NULL means

NULL = "no value / unknown". It is not 0 and not an empty string.

- suspicious_start is NULL when the video had no suspicious part.
- user_feedback is NULL until the user gives feedback.
- In SQL, test with IS NULL , never = NULL (which never matches):

```
SELECT * FROM analyses WHERE user_feedback IS NULL;
```

8. Design choices (and why)

Choice	Why
PostgreSQL, hosted on Neon	Free, managed, and the app's server (Cloud Run) has no permanent disk, so a local SQLite file would be wiped
Raw SQL, no ORM	Few queries, easy to read and learn from

Choice	Why
Tables created on startup (<code>IF NOT EXISTS</code>)	No migration tool needed for 2 tables
Two separate tables, not one	Uploads and live sessions have different columns (a live session has no filename or suspicious window; it has a duration instead)
<code>UNION ALL</code> for history	Shows both kinds in one list without merging the tables
Per-frame scores as <code>JSONB</code>	Always read/written together with their row; saves a join
Video on disk, path in DB	Databases are bad at storing big video files

9. Handy commands to explore the database yourself

Connect with `psql` (the Postgres command-line client):

```
psql "postgresql://USER:PASSWORD@HOST/dbname?sslmode=require"
```

`psql` shortcuts (start with a backslash, no semicolon needed):

Command	Does
<code>\l</code>	List databases
<code>\dt</code>	List tables
<code>\d analyses</code>	Show a table's columns, types and constraints
<code>\x</code>	Toggle vertical output (nice for wide rows)
<code>\q</code>	Quit

Practice queries (read-only, safe to run):

```

-- How many videos analysed?
SELECT COUNT(*) FROM analyses;

-- Latest 5 analyses
SELECT id, filename, status, fake_probability, created_at
FROM analyses
ORDER BY created_at DESC
LIMIT 5;

-- How many of each verdict? (GROUP BY)
SELECT status, COUNT(*) AS total
FROM analyses
GROUP BY status;

-- Only the likely fakes
SELECT filename, fake_probability
FROM analyses
WHERE status = 'HIGH_RISK';

-- Average live-session length
SELECT AVG(duration_seconds) FROM live_sessions;

-- Number of scored frames inside the JSONB list
SELECT id, filename, jsonb_array_length(frame_scores) AS frames
FROM analyses;

-- Unpack the JSONB list into rows (one row per frame)
SELECT a.id,
       (f->>'timestamp')::float      AS t,
       (f->>'fake_probability')::float AS p
FROM analyses a, jsonb_array_elements(a.frame_scores) AS f
WHERE a.id = 1;

```

JSON operators: `->` gets a JSON value, `->>` gets it as text, and `::float` **casts** (converts) text to a number.

Be careful in `psql` with `UPDATE`, `DELETE`, `DROP` and `TRUNCATE` — on Neon they change the real project data.

10. Quick glossary

Term	Meaning
DBMS	Database Management System — software that stores and manages data (here: PostgreSQL)
Table / relation	Data organised in rows and columns
Row / tuple / record	One entry in a table
Column / attribute / field	One property every row has

Term	Meaning
Schema	The structure of the database: tables, columns, types, rules
Primary key	Column that uniquely identifies each row
Foreign key	Column pointing to a primary key in another table (not used here)
Index	Lookup structure that makes searches faster (auto-created for primary keys)
DDL	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> — change structure
DML	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> — work with data
Transaction	Group of changes applied all-or-nothing
Commit / rollback	Save / undo a transaction
Cursor	Object used to run queries and read results
Driver	Library that lets a language talk to a DB (<code>psycopg2</code>)
ORM	Tool that maps tables to classes (not used here)
Migration	Script that changes an existing schema safely
SQL injection	Attack that sneaks SQL in through user input; stopped by placeholders
Subquery	A query inside another query
<code>UNION ALL</code>	Stack results of two <code>SELECT</code> s, keeping duplicates
<code>NULL</code>	Missing / unknown value
JSONB	Postgres type for storing JSON efficiently
Normalization	Organising tables to avoid repeated data; the <code>JSONB</code> lists are a deliberate small exception

11. Likely viva / interview questions

Which database and why? PostgreSQL on Neon — free, hosted, relational, has `JSONB` and `RETURNING` ; the deploy server has no permanent disk.

How many tables? Two: `analyses` and `live_sessions` . No relationship between them.

What is the primary key? `id` (`SERIAL`) in each table.

How do you prevent SQL injection? Parameterized queries with `%s` placeholders; values are passed separately to `cur.execute()` .

How are tables created? `CREATE TABLE IF NOT EXISTS` on server startup, plus `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` for columns added later.

Why is the video not stored in the database? Large binary files bloat the database and slow it down; the file goes on disk and the DB stores its path.

Why `JSONB` for frame scores instead of another table? The list is always used as a whole with its parent row; `JSONB` avoids a join and an extra insert per frame. Trade-off: slightly less normalized.

What does RETURNING * do? Returns the inserted/updated row in the same statement, including auto-generated values like `id` and `created_at` .

UNION vs UNION ALL ? `UNION` removes duplicate rows (extra work); `UNION ALL` keeps all rows. The history query uses `UNION ALL` .

What happens if the server crashes mid-insert? The transaction is never committed, so it is rolled back — no half-saved row (atomicity).